

Documento de Diseño Arquitectónico

Proyecto: Data Warehouse Educativo — MINEDUC Ecuador

Nombre: Jean Carlos Cuenca Alvarado ---Fecha: 19/05/2026

I. JUSTIFICACIÓN DEL DISEÑO

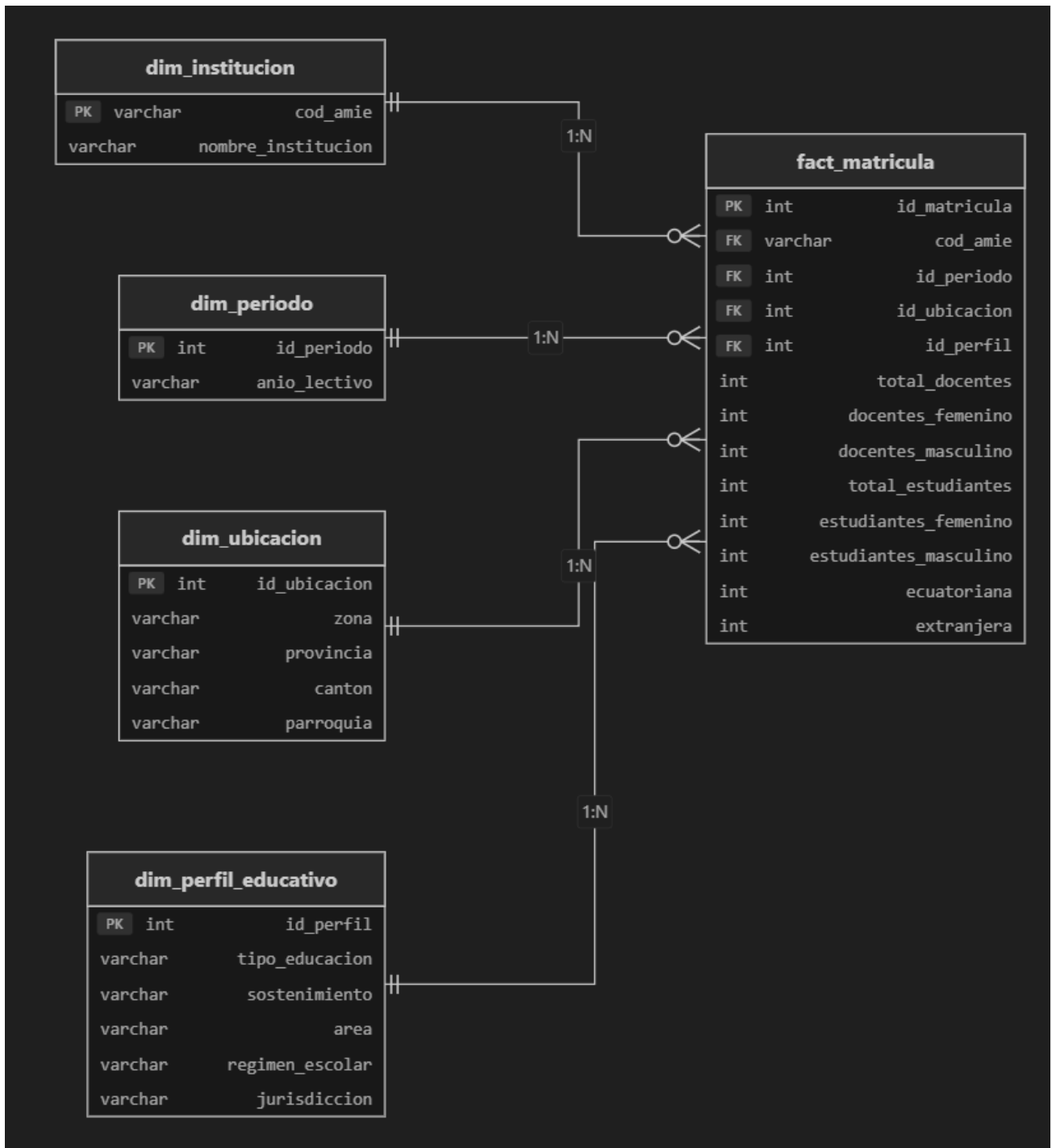
Para este proyecto de ingeniería de datos se optó por implementar un Modelo en Estrella (Star Schema). El origen de la información consistía en un archivo CSV plano y desnormalizado, comúnmente denominado 'sábana de datos'. Este formato es altamente ineficiente para el análisis analítico interactivo, dado que genera una redundancia masiva de texto; por ejemplo, la repetición de cadenas correspondientes a provincias, cantones y parroquias se produce millones de veces en disco.

Al segmentar la sábana transaccional e implementar un esquema en estrella optimizado, se obtienen los siguientes beneficios arquitectónicos fundamentales:

Beneficio	Descripción
Optimización de Almacenamiento	Los atributos de texto con alta longitud (nombres de instituciones, jerarquías geográficas, perfiles operativos) se indexan y almacenan una única vez en tablas de Dimensiones. La tabla de hechos solo contiene llaves foráneas numéricas y métricas aditivas puras.
Consultas Analíticas de Alta Velocidad	Los motores de BI modernos (Power BI, Metabase, Tableau) están optimizados para resolver operaciones de agregación (SUM, COUNT, AVG) mediante JOINs directos en esquemas en estrella, reduciendo los tiempos de respuesta a milisegundos.

II. DIAGRAMA ENTIDAD-RELACIÓN (ER DIAGRAM)

La arquitectura implementada consta de cuatro tablas de dimensiones independientes que rodean y contextualizan a una única tabla de hechos centralizada, consolidando la integridad referencial y las cardinalidades de uno a muchos (1:N) requeridas por las mejores prácticas corporativas de modelado dimensional.



III. ORQUESTADOR DEL PIPELINE ETL

Para la limpieza y transformación del archivo CSV se desarrolló un script orquestador denominado pipeline.py. Este módulo actúa como punto de entrada principal del proceso ETL (Extracción, Transformación y Carga), y su función es coordinar, de forma ordenada, la ejecución secuencial de tres módulos independientes: extract.py, transform.py y load.py.

Esta arquitectura modular ofrece una ventaja clave en términos de mantenibilidad: si en el futuro el origen de datos cambia o la base de datos migra a otro sistema, basta con modificar únicamente el módulo afectado, sin necesidad de alterar el

resto del flujo. De esta manera, el código resultante es escalable, comprensible y fácil de mantener. A continuación se documenta el funcionamiento de cada módulo.

A. Extracción de Datos (extract.py)

Dado que la fuente de información es un registro gubernamental ecuatoriano, el archivo CSV presentaba caracteres propios del español, tales como tildes y la letra 'ñ'. Para evitar errores de decodificación durante la lectura, se configuró el estándar de codificación latin-1 (Windows-1252), ampliamente compatible con los caracteres del idioma español.

Adicionalmente, el script fue diseñado para detectar dinámicamente distintos separadores de columnas. Esta característica garantiza una ingesta de datos robusta ante variaciones de formato en el archivo fuente, previniendo fallos en etapas posteriores del pipeline.

B. Transformación y Modelado en Memoria (transform.py)

El principal desafío de calidad de datos se encontró en las métricas numéricas, tales como el número de docentes y estudiantes. Estas columnas estaban almacenadas como cadenas de texto contaminadas con espacios en blanco y comas utilizadas como separadores de miles. Para resolverlo, se aplicaron operaciones de limpieza de cadenas (.strip() y .replace()), seguidas de una conversión segura hacia el tipo de dato entero (Integer), garantizando así la integridad numérica de los datos.

Posteriormente, se realizó la transformación estructural hacia el Modelo en Estrella directamente en memoria RAM, utilizando DataFrames de Pandas. Mediante operaciones de drop_duplicates y merge, la 'sábana de datos' desnormalizada fue segmentada en cuatro tablas dimensionales y una tabla de hechos central. Como medida adicional, todos los nombres de columnas fueron estandarizados a minúsculas para prevenir errores de sensibilidad a mayúsculas (Case Sensitivity) propios del motor PostgreSQL.

C. Carga Optimizada a la Base de Datos (load.py)

La tabla de hechos resultante del proceso de transformación contiene aproximadamente 442,716 registros. Intentar cargar este volumen de datos en una sola transacción podría saturar la memoria del servidor o provocar un tiempo de espera excedido (Timeout) en la base de datos. Para evitar este problema, se implementó una estrategia de inserción por bloques (chunking).

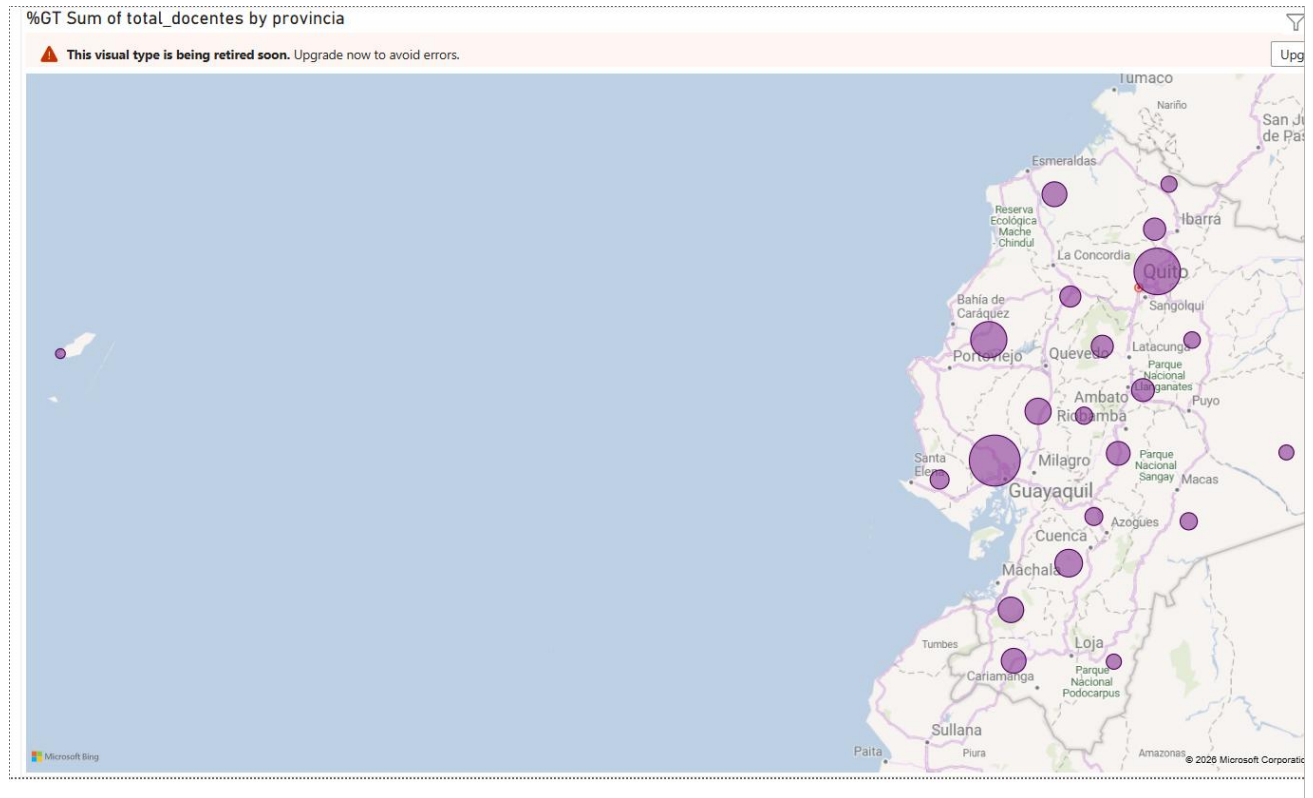
Concretamente, se configuró el motor SQLAlchemy con los parámetros method='multi' y chunksize=1000, lo que permite dividir las inserciones en lotes de mil registros por transacción. Esta técnica redujo drásticamente los tiempos de carga y aseguró la persistencia total de los datos en el esquema amie_dw de PostgreSQL, sin comprometer la estabilidad del servidor.

IV. VISUALIZACIÓN EN DASHBOARD (POWER BI)

Una vez cargados los datos en PostgreSQL, se realizaron consultas SQL estructuradas sobre el Data Warehouse para alimentar las visualizaciones en Power BI. A continuación se presentan las principales consultas implementadas.

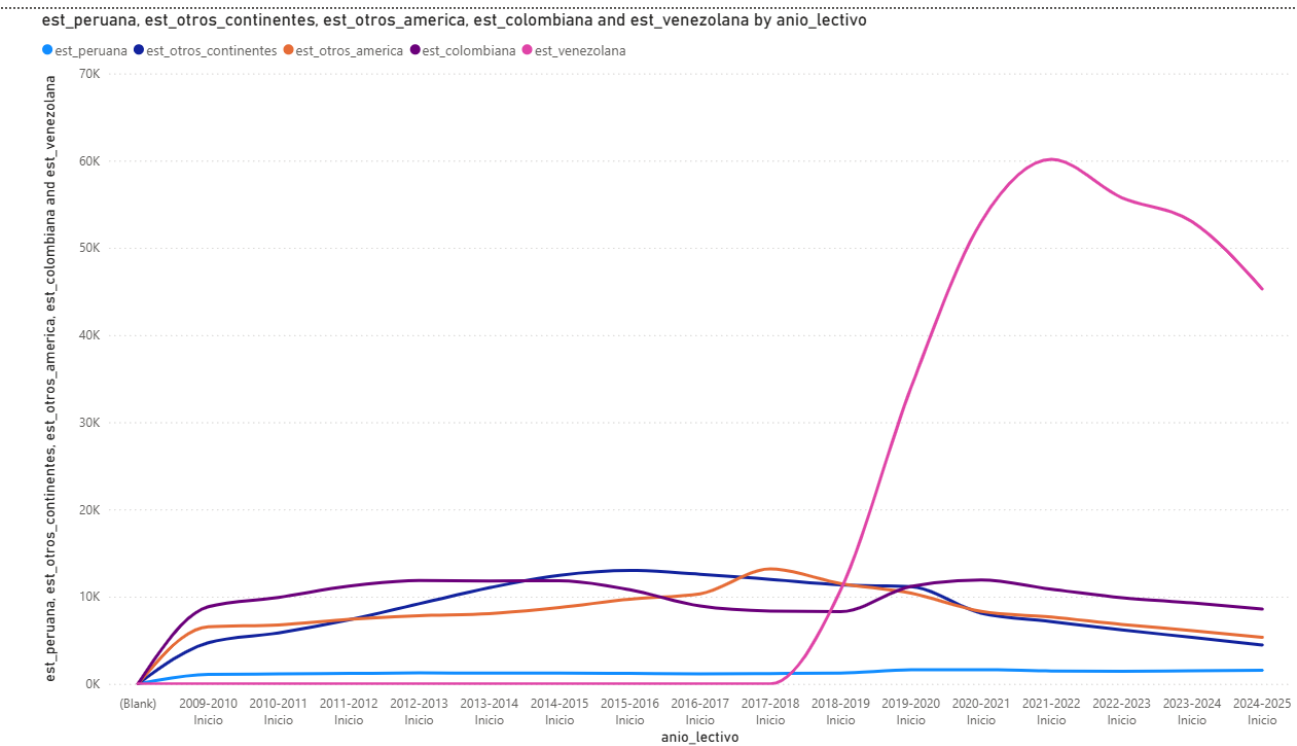
Consulta 1 — Matrícula por Provincia y Año Lectivo

Esta consulta obtiene el total de estudiantes matriculados, desglosados por género, provincia y año lectivo:



```
SELECT
  p.anio_lectivo,
  u.provincia,
  SUM(f.total_estudiantes) AS total_estudiantes,
  SUM(f.est_femenino)      AS estudiantes_femenino,
  SUM(f.est_masculino)     AS estudiantes_masculino
FROM amie_dw.fact_matricula f
JOIN amie_dw.dim_periodo   p ON f.id_periodo = p.id_periodo
JOIN amie_dw.dim_ubicacion u ON f.id_ubicacion = u.id_ubicacion
GROUP BY p.anio_lectivo, u.provincia
ORDER BY p.anio_lectivo, total_estudiantes DESC;
```

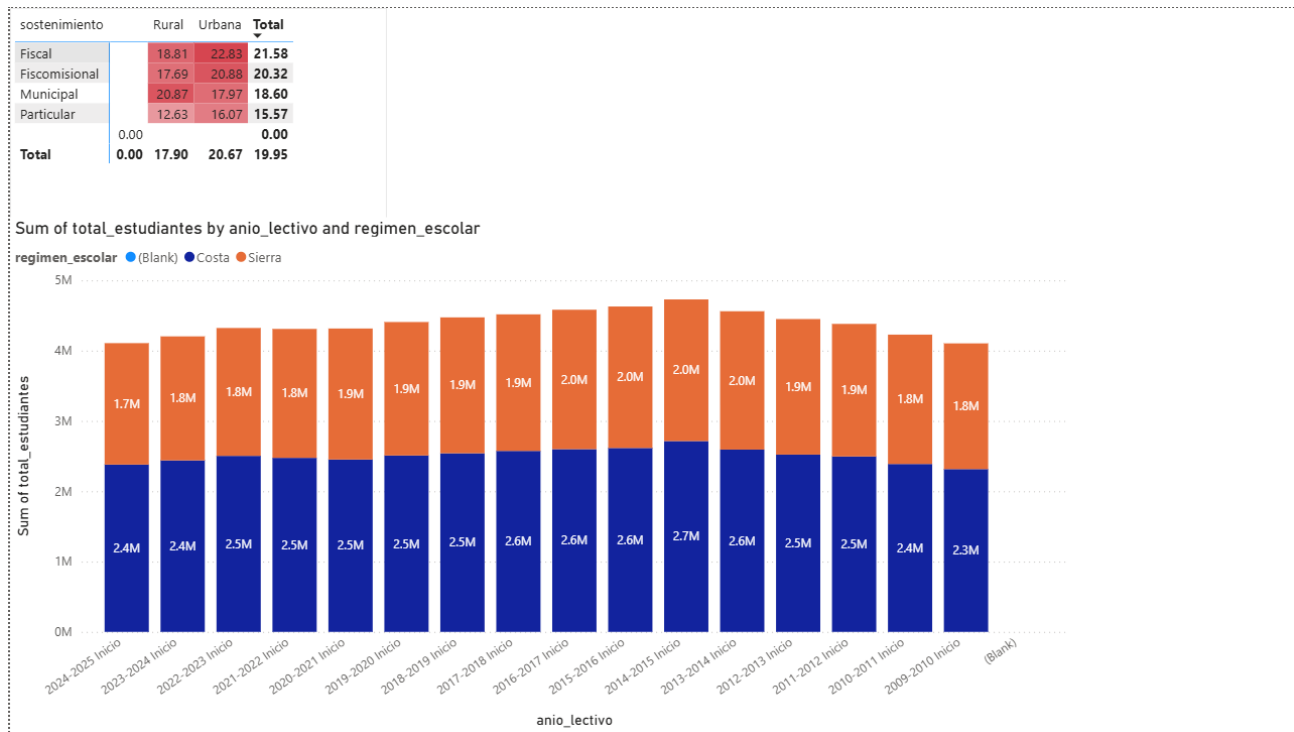
Consulta 2 — Composición de Estudiantes por Nacionalidad a lo Largo del Tiempo



Esta consulta analiza la distribución histórica de estudiantes según su nacionalidad de origen:

```
SELECT
    p.anio_lectivo,
    SUM(f.est_ecuatoriana)      AS ecuatorianos,
    SUM(f.est_colombiana)      AS colombianos,
    SUM(f.est_venezolana)      AS venezolanos,
    SUM(f.est_peruana)         AS peruanos,
    SUM(f.est_otros_america)    AS otros_america,
    SUM(f.est_otros_continentes) AS otros_continentes
FROM amie_dw.fact_matricula f
JOIN amie_dw.dim_periodo p ON f.id_periodo = p.id_periodo
GROUP BY p.anio_lectivo
ORDER BY p.anio_lectivo;
```

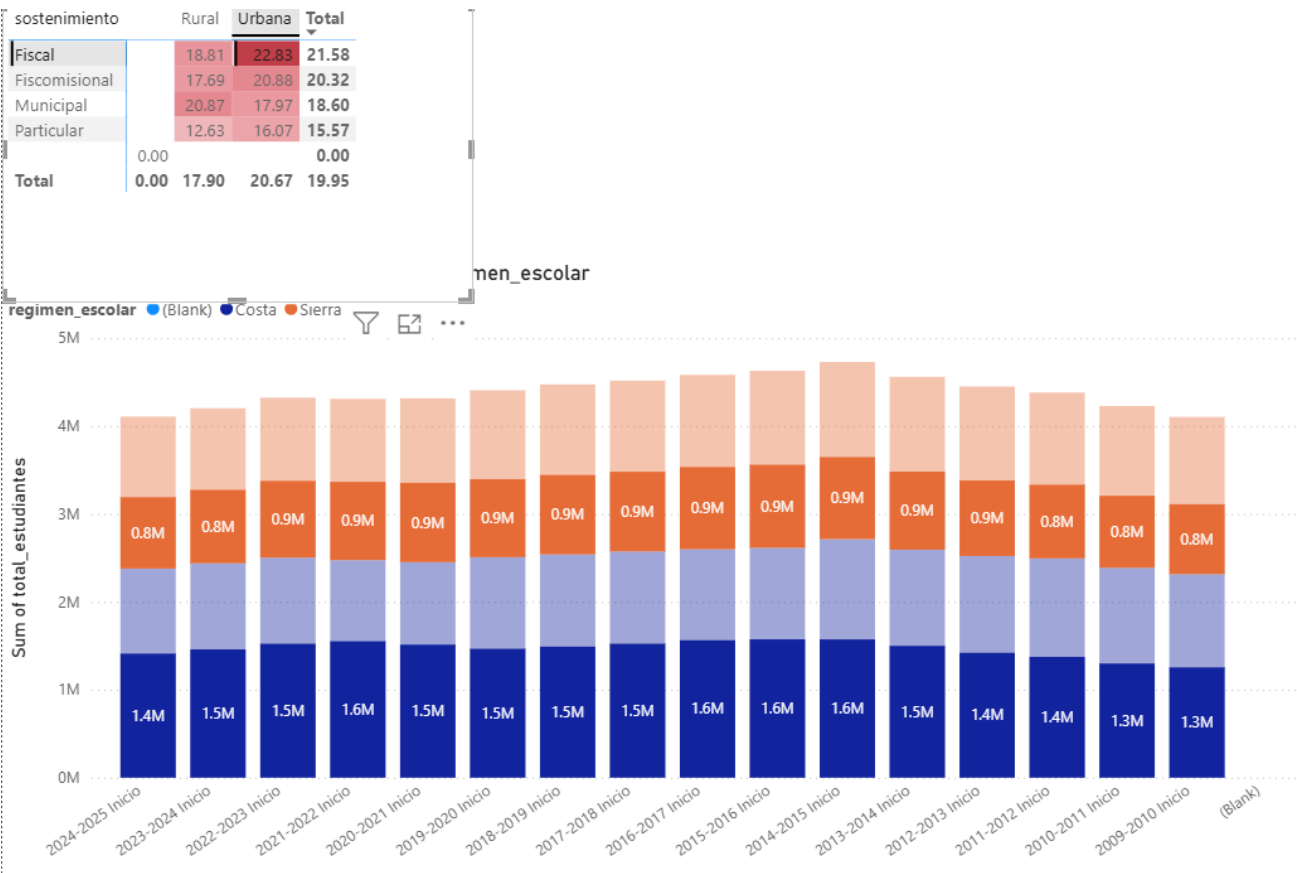
Consulta 3 — Ratio Estudiante/Docente por Tipo de Institución y Área Geográfica



Esta consulta identifica la cantidad de estudiantes según el tipo de institución y su ubicación (rural o urbana), permitiendo determinar históricamente qué tipo de institución concentra la mayor presencia estudiantil, diferenciando entre la región Sierra y la Costa:

```
SELECT
    pe.sostenimiento,
    pe.area,
    p.anio_lectivo,
    SUM(f.total_estudiantes) AS total_estudiantes,
    SUM(f.total_docentes) AS total_docentes,
    -- NULLIF evita errores de división por cero
    ROUND(
        SUM(f.total_estudiantes)::numeric /
        NULLIF(SUM(f.total_docentes), 0), 2
    ) AS ratio_est_docente
FROM amie_dw.fact_matricula f
JOIN amie_dw.dim_periodo p ON f.id_periodo = p.id_periodo
JOIN amie_dw.dim_perfil_educativo pe ON f.id_perfil = pe.id_perfil
GROUP BY pe.sostenimiento, pe.area, p.anio_lectivo
ORDER BY p.anio_lectivo, pe.sostenimiento, pe.area;
```

Consulta 4 — *Estudiantes por Régimen Escolar*



```
SELECT
    pe.regimen_escolar,
    p.anio_lectivo,
    SUM(f.total_estudiantes) AS total_estudiantes
FROM amie_dw.fact_matricula f
JOIN amie_dw.dim_periodo p ON f.id_periodo = p.id_periodo
JOIN amie_dw.dim_perfil_educativo pe ON f.id_perfil = pe.id_perfil
GROUP BY pe.regimen_escolar, p.anio_lectivo
ORDER BY p.anio_lectivo, pe.regimen_escolar;
```